

C++ Sliding Window Solution

Here is the **C++ solution** using a sliding window. The key observation is that every forbidden string has length at most **10**, which lets us check only the last 10 characters at each position.

```
C++
class Solution {
public:
    int longestValidSubstring(string word, vector<string>& forbidden) {
        unordered_set<string> bad(forbidden.begin(), forbidden.end());

        int n = word.size();
        int left = 0;
        int ans = 0;

        for (int right = 0; right < n; right++) {
            // Check substrings ending at 'right'
            // Maximum forbidden length is 10
            string cur = "";

            for (int i = right; i >= max(left, right - 9); i--) {
                cur = word[i] + cur;

                if (bad.count(cur)) {
                    // This forbidden substring starts at i
                    // Therefore left must move past i
                    left = i + 1;
                    break;
                }
            }

            ans = max(ans, right - left + 1);
        }

        return ans;
    }
};
```

Why only 10 characters?

Because the constraint says:

```
1 <= forbidden[i].length <= 10
```

So if we're currently at index **right**, a forbidden substring ending at **right** can only start within:

```
right - 9
```

For example:

```
word = "cbaaaaabc"
      ↑
      right
```

If **right** is at the last **c**, we only need to check substrings of length:

```
1, 2, 3, ... 10
```

ending at that **c**.

This gives roughly **$O(10 \times n)$** time, which is effectively **$O(n)$** for $n \leq 10^5$.

The important part is:

```
C++
left = i + 1;
```

If **word[i...right]** is forbidden, then **any substring starting at or before i and ending at right is invalid**, so we move **left** to **i + 1**.

Let's focus only on why we use:

```
C++
right - 9
```

and not check further left.

Example

Suppose:

```
word = "abcdefghijlm"
```

Indices:

```
a b c d e f g h i j k l m
0 1 2 3 4 5 6 7 8 9 10 11 12
```

Let's say:

```
C++
right = 12; // 'm'
```

Now we are trying to find forbidden substrings that **end at index 12**.

Since every forbidden string has length at most **10**:

```
max forbidden length = 10
```

the longest possible forbidden substring ending at 12 would be:

```
start = 12 - 10 + 1
       = 3
```

which gives:

```
d e f g h i j k l m
3                12
```

length = 10.

What substrings do we check?

The loop:

```
C++
for(int i = right; i >= right - 9; i--)
```

checks:

i = 12 -> "m"	length 1
i = 11 -> "lm"	length 2
i = 10 -> "klm"	length 3
i = 9 -> "jklm"	length 4
i = 8 -> "ijklm"	length 5
i = 7 -> "hijklm"	length 6
i = 6 -> "ghijklm"	length 7
i = 5 -> "fghijklm"	length 8
i = 4 -> "efghijklm"	length 9
i = 3 -> "defghijklm"	length 10

After that we stop.

Why don't we check i = 2?

That would create:

```
cdefghijklm
2          12
```

Length:

```
12 - 2 + 1 = 11
```

But forbidden strings can never have length 11.

So:

```
length > 10
```

means it is impossible for that substring itself to be in **forbidden**.

Therefore checking it is useless.

Another small example

Suppose:

```
C++
```

```
forbidden = {"abcde"} // length 5
```

and

```
word = "xyzabcde"
```

At:

```
C++
right = 7 // 'e'
```

We check:

```
"e"
"de"
"cde"
"bcde"
"abcde" <-- found
```

Notice we only needed to go back 4 positions.

The code goes back up to 9 positions because **10 is the maximum possible forbidden length**.

Why right - 9 and not right - 10?

Length formula:

```
length = right - start + 1
```

For length 10:

```
10 = right - start + 1
start = right - 9
```

That's why:

```
C++
for (int i = right; i >= right - 9; i--)
```

checks exactly all lengths:

```
1, 2, 3, ..., 10
```

and nothing longer.

Yes. There are a few cleaner ways to write that loop. The one I recommend for understanding is to **build the substring using substr()** instead of doing:

```
C++
cur = word[i] + cur;
```

Cleaner version

```
C++
for (int len = 1; len <= 10 && len <= right - left + 1; len++) {
    string cur = word.substr(right - len + 1, len);
    if (bad.count(cur)) {
        left = right - len + 2;
        break;
    }
}
```

How this works

Suppose:

```
word = "abcdef"
right = 5
```

We're at:

```
a b c d e f
0 1 2 3 4 5
      ↑
    right
```

The loop checks:

```
C++
len = 1
```

```
f
```

because:

```
C++
word.substr(5, 1)
```

Then:

```
C++
len = 2
```

```
ef
```

because:

```
C++
word.substr(4, 2)
```

Then:

```
C++
len = 3
```

```
def
```

and so on:

```
len = 1 → "f"
len = 2 → "ef"
len = 3 → "def"
len = 4 → "cdef"
...
len = 10
```

So it's doing exactly the same thing, but the **length-based approach is easier to understand**.

Full code

```
C++
class Solution {
public:
    int longestValidSubstring(string word, vector<string>& forbidden) {
        unordered_set<string> bad(forbidden.begin(), forbidden.end());
        int n = word.size();
        int left = 0;
        int ans = 0;
        for (int right = 0; right < n; right++) {
            for (int len = 1; len <= 10 && len <= right - left + 1; len++) {
                string cur = word.substr(right - len + 1, len);
                if (bad.count(cur)) {
                    left = right - len + 2;
                    break;
                }
            }
            ans = max(ans, right - left + 1);
        }
        return ans;
    }
};
```

However, for this problem, I actually prefer the original version once you understand it, because it avoids repeatedly creating substrings with `substr()` and is a little more efficient.